

ENHANCED EMERGENCY RESPONSE SYSTEM

DONE BY:

22I248-NAVEEN S S

23I431-CHARAN K

23I433-SACHIN KATHIK V

23I434-SIVARAMAN S

23I435-VIJAYABASKAR

TABLE OF CONTENTS

S.NO	TOPIC	PAGE NUMBER
01	ABSTRACT	03
02	INTRODUCTION	04
03	COMPONENTS DESCRIPTION	05 - 07
04	PROPOSED METHODOLOGY	08 – 11
05	CODE	12 - 15
06	CONNECTION SET-UP	16 - 19
07	SOFTWARE DEVELOPMENT	20
08	TESTING AND DEBUGGING	21
09	APPLICATION	22
10	RESULT	23
11	REFERENCES	24

ABSTRACT

The Enhanced Emergency Response System (EERS) is an IoT-based solution designed to detect and respond to various environmental hazards, such as fires, gas leaks, and abnormal temperature fluctuations, in real time. The system incorporates a network of environmental sensors (gas, smoke, temperature, and fire), microcontrollers (NodeMCU ESP8266), and a fog computing architecture for local data processing and immediate decision-making. This architecture reduces response time by processing critical data locally, ensuring that real-time alerts are sent instantly. The system also integrates cloud services for long-term data storage, advanced analytics, and remote notifications, providing a detailed history of events for further analysis. Users can access sensor data, receive alerts, and monitor real-time conditions through a web or mobile application, making the system accessible from anywhere. With a focus on scalability, the EERS can be applied to various environments, from homes to large industrial settings, while providing enhanced emergency preparedness and response. The integration of Wi-Fi enables seamless communication between the hardware and cloud, ensuring reliable monitoring and decision-making capabilities during critical situations. Additionally, its modular design allows for easy expansion, adding more sensors or functionalities as needed, making it a versatile solution for evolving safety requirements.

CHAPTER 1

INTRODUCTION

Emergencies such as fires, gas leaks, and hazardous environmental conditions pose significant risks to human lives and property. Traditional safety systems often depend on manual intervention and isolated detection methods, which can delay critical response times and lead to serious consequences. In today's technological landscape, leveraging the Internet of Things (IoT) is essential for creating efficient solutions for emergency detection and response.

The Enhanced Emergency Response System (EERS) is an innovative IoT-based solution designed to address these challenges by detecting environmental hazards in real time and enabling immediate responses. By combining various sensors—such as gas, smoke, temperature, and fire—with NodeMCU (ESP8266) microcontrollers, the system effectively monitors and responds to emergencies as they occur. The integration of fog computing allows for rapid decision-making, ensuring timely alerts and minimizing potential damage.

Additionally, the system connects to cloud services for long-term data storage, analysis, and notifications, maintaining a comprehensive record of historical data for trend analysis. Users can access real-time updates and monitor conditions through a web or mobile application, facilitating informed decision-making from anywhere.

The EERS is versatile and scalable, making it suitable for a wide range of environments, including homes, offices, factories, and public spaces. Its modular design allows for easy integration of additional sensors or features, adapting to evolving safety requirements. Reliable Wi-Fi connectivity ensures seamless communication between hardware components, the cloud, and the user interface.

By utilizing IoT, fog computing, and cloud technologies, the Enhanced Emergency Response System provides immediate local responses to hazards and offers valuable insights for improving safety measures. This proactive solution enhances emergency preparedness by equipping users to effectively detect, monitor, and respond to emergencies, thereby minimizing risks and ensuring a safer environment for all.

CHAPTER 2

COMPONENTS DESCRIPTION

2.1 NODEMCU (ESP8266):

The [NodeMCU](#) is a [microcontroller](#)[1] with built-in Wi-Fi, serving as the system's core. It collects data from sensors, processes it, and communicates with cloud platforms or mobile devices for remote monitoring. Its Wi-Fi capability enables real-time alerts, making it ideal for IoT-based emergency response systems.

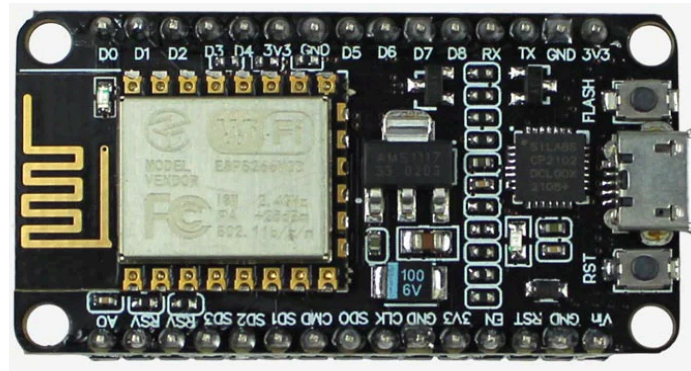


FIGURE 2.1 NODEMCU

2.2 TEMPERATURE SENSOR(DHT11):

The [DHT11 sensor](#)[2] measures temperature and humidity, detecting sudden spikes in temperature that could indicate fire hazards. By monitoring environmental conditions, it helps provide early warnings during potential fire emergencies. It is cost-effective, although its accuracy is limited compared to more advanced sensors.

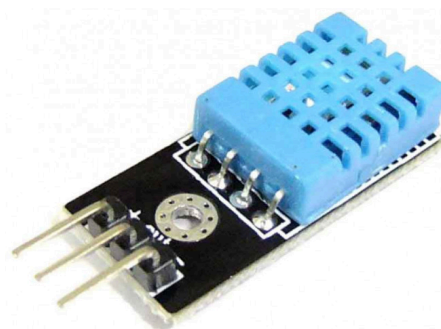


FIGURE 2.2 DHT11

2.3 IR SENSOR:

The [IR sensor](#)[3] detects infrared radiation emitted by flames, allowing it to quickly identify the presence of fire. This sensor enhances the system's ability to respond swiftly to fire hazards.

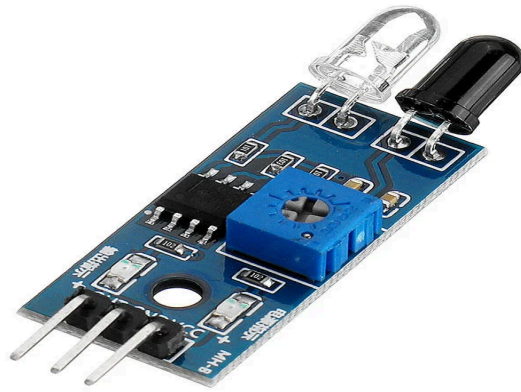


FIGURE 2.3 IR SENSOR

2.4 GAS SENSOR(MQ 2):

The [MQ-2 sensor](#)[4] detects smoke and combustible gasses such as propane and methane, helping to identify gas leaks or fires. It serves as an early warning system by sending real-time data to the controller, which triggers alarms if hazardous gas levels are detected.



FIGURE 2.4 MQ-2

2.5 BUZZER:

The buzzer produces an audible alarm whenever danger is detected by the system. It is triggered by the microcontroller to alert nearby individuals to hazards like fire or gas leaks, providing a critical safety feature that ensures immediate attention.



FIGURE 2.5 BUZZER

2.6 JUMPER WIRES & BREADBOARD:

Jumper wires and a breadboard allow flexible and temporary connections between the components. This setup simplifies prototyping and circuit design adjustments, enabling easy testing, troubleshooting, and modifications during the development.

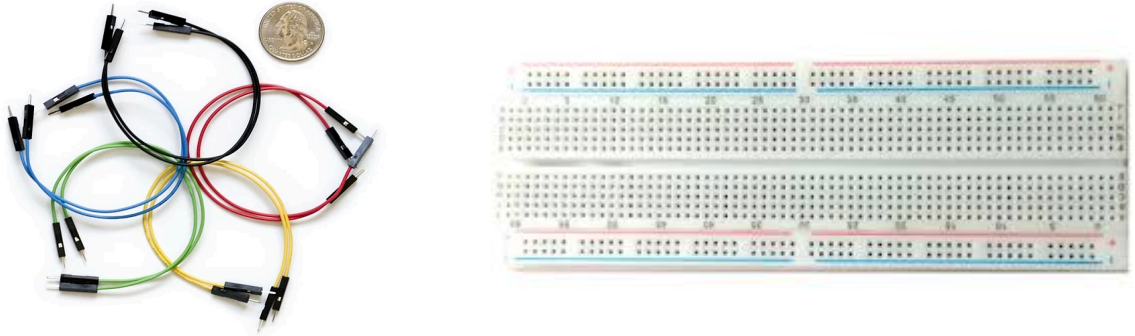


FIGURE 2.6 JUMPER WIRES & BREADBOARD

CHAPTER 3

PROPOSED METHODOLOGY

3.1 METHODS

3.1.1 HARDWARE CONNECTION SETUP

- **Sensors:** The system incorporates smoke, gas, temperature, and fire sensors connected to either the Arduino Uno or ESP8266 microcontrollers. These sensors are configured to monitor environmental conditions and send data to the microcontroller.
- **Microcontroller Programming:** we developed code for periodic data collection, processing this data to detect threshold breaches, which would indicate hazardous conditions.
- **Local Alarm System:** An alarm system (buzzer and LED lights) was integrated to provide immediate local alerts when any hazard is detected.

3.1.2 CONNECTING TO THINGSPEAK

- **Wi-Fi Connection:** The ESP8266 was connected to a Wi-Fi network for communication with ThingSpeak.
- **ThingSpeak Account Setup:** A ThingSpeak account was created, and a channel was established to log sensor data across four fields: smoke, gas, temperature, and fire detection.
- **API Integration:** The microcontroller is programmed to send data to ThingSpeak at regular intervals, enabling real-time monitoring.

3.1.3 USER INTERFACE DEVELOPMENT

- **Web Application:** A user-friendly interface was developed using HTML/CSS/JavaScript for web access and Flutter for mobile. This interface displays real-time sensor data and historical trends.
- **Dashboard:** The dashboard includes visual representations of sensor data and notifications for any threshold breaches.

3.1.4 SENDING ALERT MESSAGES VIA TWILIO

- **Twilio Setup:** A Twilio account was created to send SMS alerts. API credentials and a Twilio phone number were obtained for alert messaging.
- **Triggering Alerts:** The system is programmed to trigger alerts when sensors exceed safety thresholds. SMS alerts inform users of emergencies, providing critical details.

3.1.5 TESTING AND CALIBRATION

- **Testing Sensors:** The system was tested in controlled environments to ensure accurate detection of hazards.

- Calibration: Sensor thresholds were adjusted to balance sensitivity and accuracy, minimizing false alarms.

3.1.6 END-TO-END SYSTEM INTEGRATION

The complete system was tested to ensure smooth operation from data collection to alert generation. The integration allows for:

- Real-time sensor detection.
- Data uploaded to ThingSpeak.
- Immediate user alerts via the interface and SMS.

3.2 SYSTEM DESIGN

The design of the Distributed Emergency Management System is structured around key modules, ensuring robust hazard detection and responsive action. The system is divided into four primary modules.

3.2.1 HAZARD DETECTION MODULE:

This module includes the MQ-2 gas sensor, DHT11 temperature sensor, and IR sensor. Together, they monitor environmental conditions for gas leaks, smoke, rapid temperature increases, or infrared radiation from flames. Data from these sensors are sent to the NodeMCU for real-time processing and alert generation.

3.2.2 ALERT AND NOTIFICATION MODULE:

The system is equipped with a buzzer, which sounds an audible alert when hazardous conditions are detected. Additionally, the NodeMCU's Wi-Fi capability is utilized to send real-time alerts and sensor data to a web-based interface created using HTML, CSS, and JavaScript. This allows users to monitor and manage emergency situations remotely via the web app.

3.2.3 COMMUNICATION MODULE:

The NodeMCU (ESP8266) serves as the central controller, handling all sensor data and communicating with the web application. It ensures data transmission to the cloud for real-time monitoring and control through Wi-Fi. The web app allows users to receive alerts and view sensor status remotely, enabling proactive safety measures.

3.2.4 SYSTEM CONNECTIVITY AND USER INTERFACE MODULE:

The system interfaces with a web application designed using HTML, CSS, and JavaScript. The app provides a user-friendly interface for monitoring environmental conditions in real time, with graphical data representations and alert notifications. This module ensures seamless connectivity between the physical system and the user.

3.3 SYSTEM BLOCK DIAGRAM

- NodeMCU acts as the central processing and communication unit, interfacing with all sensors and modules.
- Sensors (MQ-2, DHT11, IR) provide real-time inputs, detecting environmental changes like gas leaks, temperature fluctuations, or fire.
- Buzzer acts as an immediate audible alert system.
- The Wi-Fi module of the NodeMCU connects to the web application, enabling remote monitoring and control.
- The Web Application displays real-time sensor data, status alerts, and allows users to manage system responses via the internet

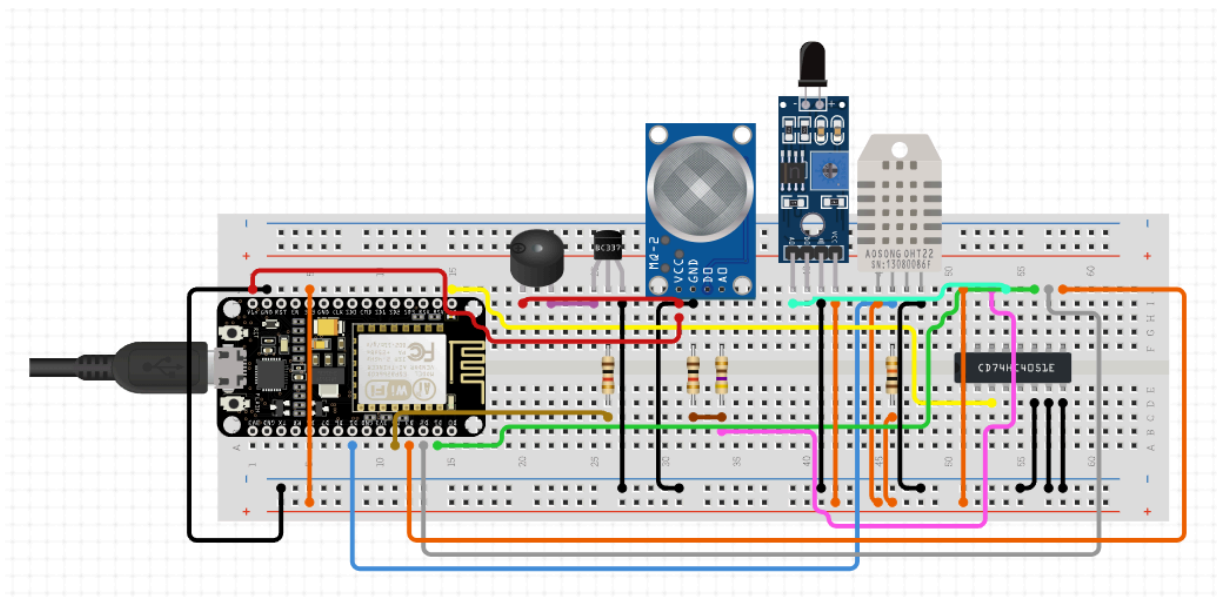


FIGURE 3.1 CIRCUIT DIAGRAM

3.4 HARDWARE INTEGRATION

3.4.1 GAS SENSOR (MQ-2) AND TEMPERATURE SENSOR (DHT11):

The MQ-2 gas sensor is connected to the NodeMCU's analog pin, detecting smoke and combustible gasses like methane and propane. The DHT11 temperature sensor is connected to a digital pin to monitor ambient temperature, detecting sudden increases indicative of fire hazards.

3.4.2 INFRARED SENSOR (IR SENSOR):

The IR sensor is interfaced with the NodeMCU via a digital pin, providing flame detection by recognizing infrared light emitted by fire. It enhances the system's ability to detect fires rapidly.

3.4.3 BUZZER:

The buzzer is connected to one of the NodeMCU's digital output pins. It is triggered to sound an audible alarm when the system detects dangerous conditions such as gas leaks, elevated temperatures, or flames.

3.4.3 WI-FI COMMUNICATION (NODEMCU):

The NodeMCU (ESP8266), with its built-in Wi-Fi, connects to a web app for remote monitoring and control. It sends real-time sensor data to the web application, enabling users to receive alerts and monitor conditions from any location.

3.4.4 POWER SUPPLY:

The system is powered by a regulated 5V power supply connected to the NodeMCU. The power is distributed efficiently across all sensors and the buzzer, ensuring continuous operation and real-time monitoring of hazardous conditions.

CHAPTER 4

CODE

```
#include <ESP8266WiFi.h>
#include <WiFiClient.h>
#include <ESP8266HTTPClient.h>
#include <DHT.h>

// Define pins for sensors and outputs
#define DHTPIN D4           // Pin where the DHT11 is connected
#define DHTTYPE DHT11      // DHT11 sensor
#define BUZZER_PIN D2       // Pin for the buzzer
#define LED_PIN D5          // Pin for the LED
#define SMOKE_SENSOR_PIN A0 // Analog pin for the smoke sensor (MQ-2)
#define GAS_SENSOR_PIN A0   // Analog pin for the gas sensor (MQ-135) -
// same as smoke sensor for testing
#define IRESENSORPIN D6     // IR sensor pin

// Initialize the DHT11 sensor
DHT dht(DHTPIN, DHTTYPE);

// ThingSpeak API credentials
const char* writeAPIKey = "DPPM6BOSWIR21G86"; // Replace with your
// ThingSpeak API key
const char* server = "api.thingspeak.com";

// WiFi credentials
const char* ssid = "12341234"; // Your WiFi SSID
const char* password = "123412345"; // Your WiFi Password

// Set threshold values
const float TEMP_THRESHOLD = 30.0; // Temperature threshold in °C
const float HUMIDITY_THRESHOLD = 70.0; // Humidity threshold in %
const int MQ2_THRESHOLD = 300; // MQ-2 gas/smoke threshold (adjust
// based on calibration)

WiFiClient client;

void setup() {
    // Initialize serial communication
    Serial.begin(115200);

    // Initialize sensor and output pins
    pinMode(BUZZER_PIN, OUTPUT);
    pinMode(LED_PIN, OUTPUT);
}
```

```

pinMode(IRSENSORPIN, INPUT); // Set IR sensor pin as input

// Start DHT sensor
dht.begin();

// Connect to WiFi
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting to WiFi...");
}
Serial.println("Connected to WiFi");
}

void loop() {
    // Read temperature and humidity from DHT11 sensor
    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

    // Read values from smoke sensor (MQ-2)
    int smokeLevel = analogRead(SMOKE_SENSOR_PIN);

    // Read IR sensor for object detection
    int irValue = digitalRead(IRSENSORPIN);
    int objectDetected = (irValue == LOW) ? 1 : 0; // 1 for detected, 0 for
not detected

    // Print sensor data to the serial monitor
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" °C");

    Serial.print("Humidity: ");
    Serial.print(humidity);
    Serial.println(" %");

    Serial.print("Smoke Level: ");
    Serial.println(smokeLevel);

    Serial.print("Object Detection: ");
    Serial.println(objectDetected ? "Object detected" : "No object
detected");

    // Check thresholds and take action

```

```

    if (temperature > TEMP_THRESHOLD || humidity > HUMIDITY_THRESHOLD ||
smokeLevel > MQ2_THRESHOLD) {
        Serial.println("WARNING: Threshold exceeded!");
        digitalWrite(BUZZER_PIN, HIGH);
        digitalWrite(LED_PIN, HIGH);

        // Send data to ThingSpeak
        sendToThingSpeak(temperature, humidity, smokeLevel, objectDetected);
    } else {
        digitalWrite(BUZZER_PIN, LOW);
        digitalWrite(LED_PIN, LOW);
    }

    // Wait for 2 seconds before the next reading
    delay(2000);
}

void sendToThingSpeak(float temperature, float humidity, int smokeLevel,
int objectDetected) {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;

        // Build URL for the GET request
        String url = "http://" + String(server) + "/update?api_key=" +
String(writeAPIKey) +
            "&field1=" + String(temperature) +
            "&field2=" + String(humidity) +
            "&field3=" + String(smokeLevel) +
            "&field4=" + String(objectDetected); // New field for
object detection

        // Print URL to serial for debugging
        Serial.print("Sending data to URL: ");
        Serial.println(url);

        // Send the request to ThingSpeak
        http.begin(client, url);
        http.setTimeout(10000); // Set timeout to 10 seconds
        int httpResponseCode = http.GET();

        // Check for HTTP errors
        if (httpResponseCode > 0) {
            Serial.print("ThingSpeak Response code: ");
            Serial.println(httpResponseCode);
        }
    }
}

```

```
    if (httpResponseCode == 200) {  
        Serial.println("Data sent successfully to ThingSpeak!");  
    } else {  
        Serial.println("Error: Non-200 response from ThingSpeak.");  
    }  
} else {  
    Serial.print("Error sending data to ThingSpeak: ");  
    Serial.println(http.errorToString(httpResponseCode));  
}  
http.end(); // Free resources  
} else {  
    Serial.println("WiFi not connected.");  
}  
}
```

CHAPTER 5

CONNECTION

5.1 COMPLETE SETUP

The hardware setup involves connecting sensors such as DHT11 for temperature and humidity, and MQ-2 for gas detection to an ESP8266 microcontroller. The system also integrates a buzzer for immediate audible alerts

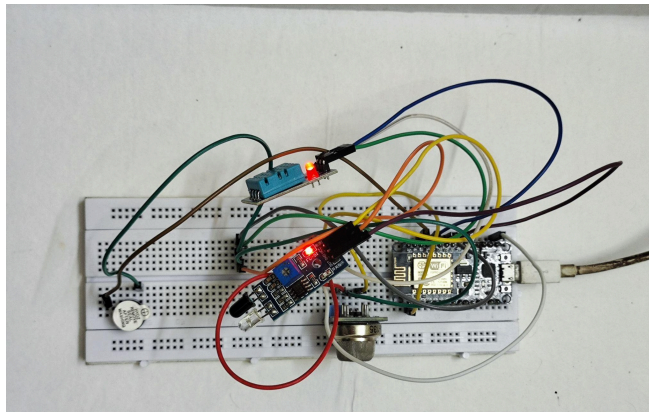


FIGURE 5.1 SETUP

5.2 THINGSPEAK

The ESP8266 microcontroller connects to Wi-Fi and sends real-time data from the DHT11 (temperature and humidity) and MQ-2 (gas) sensors to ThingSpeak. A dedicated ThingSpeak channel logs this data for continuous monitoring. Using ThingSpeak's API, the system provides real-time visualizations and alerts for potential hazards.

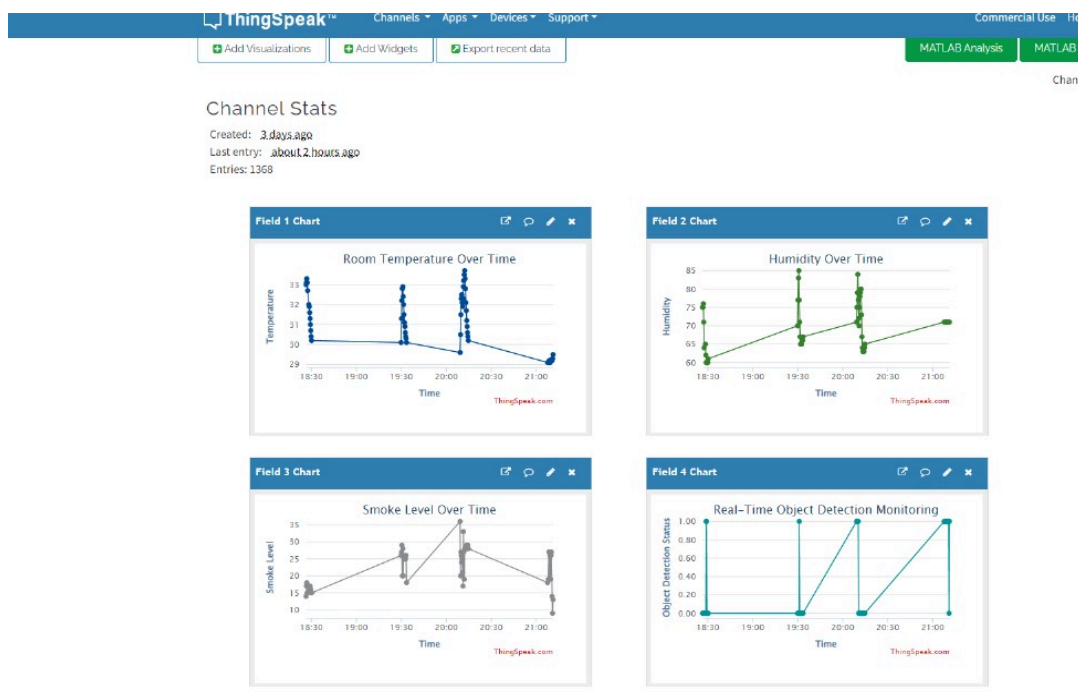


FIGURE 5.2 THINGSPEAK

5.3 HOME PAGE

The home page, built with HTML, CSS, and JavaScript, fetches real-time data from ThingSpeak using its API. It dynamically displays sensor readings like temperature, humidity, and gas levels. The dashboard provides an easy-to-understand view for monitoring environmental conditions.

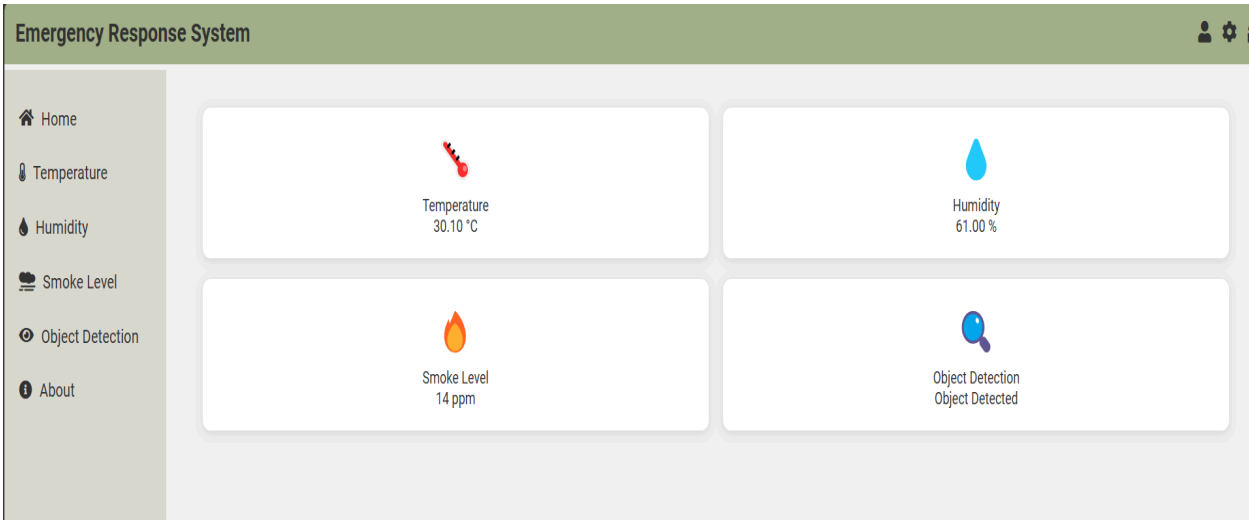


FIGURE 5.3 HOMEPAGE

5.4 INDIVIDUAL PAGE FOR SENSOR

Each sensor has a dedicated page displaying its real-time values, updated dynamically using ThingSpeak’s API. A live graph is generated on each page to visualize data trends for temperature, humidity, and gas levels. This setup allows users to monitor individual sensor performance efficiently.

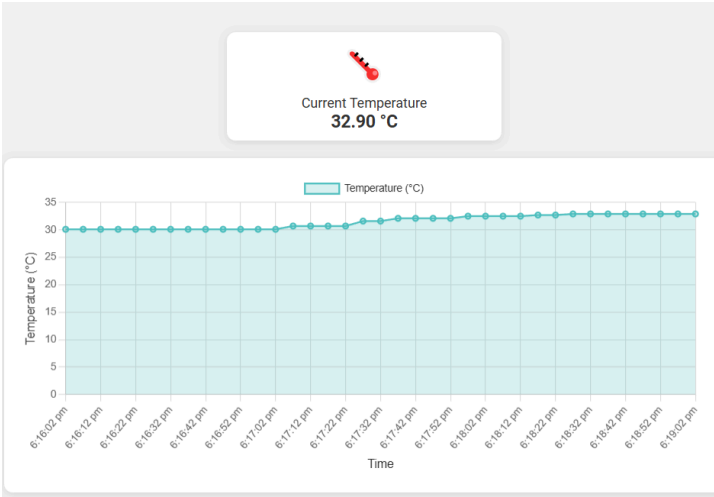


FIGURE 5.4 TEMPERATURE DATA

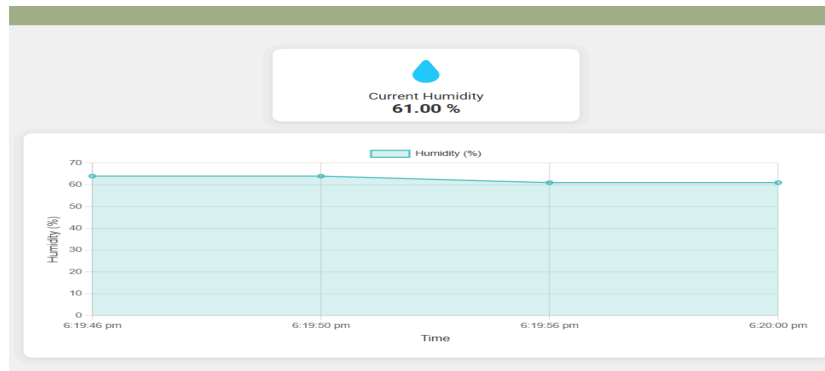


FIGURE 5.5 HUMIDITY DATA

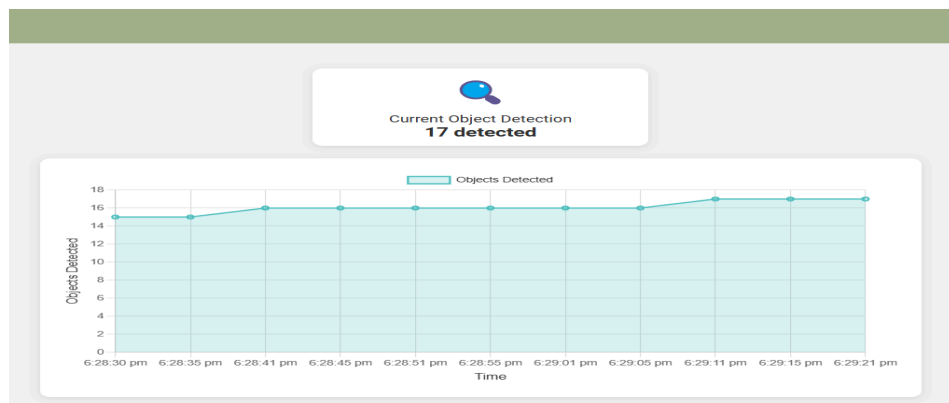


FIGURE 5.6 OBJECT DATA

5.5 ABOUT

The "About" page provides detailed information about the sensors used in the system, including DHT11 for temperature and humidity, and MQ-2 for gas detection. It also features a section introducing the team members, complete with their photos and roles in the project.

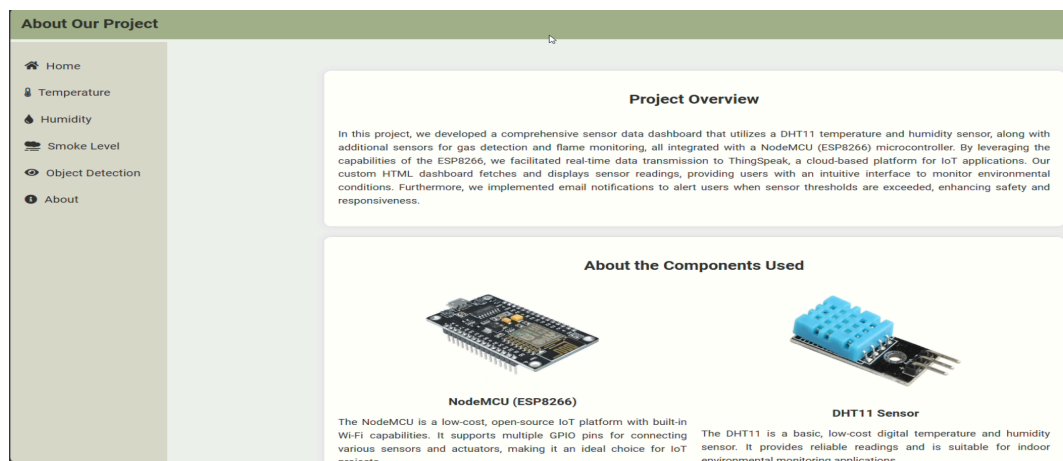
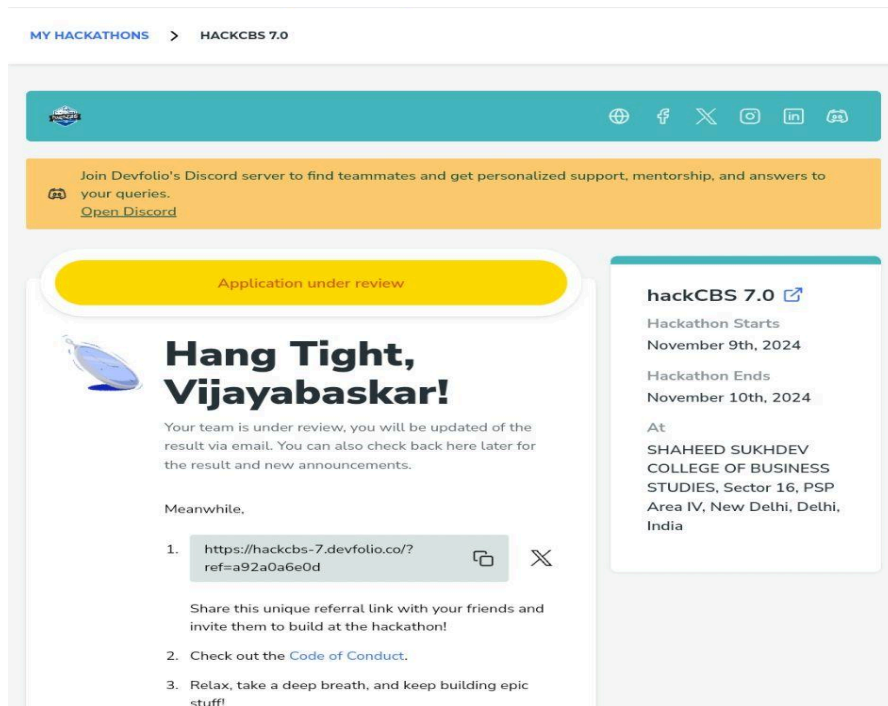
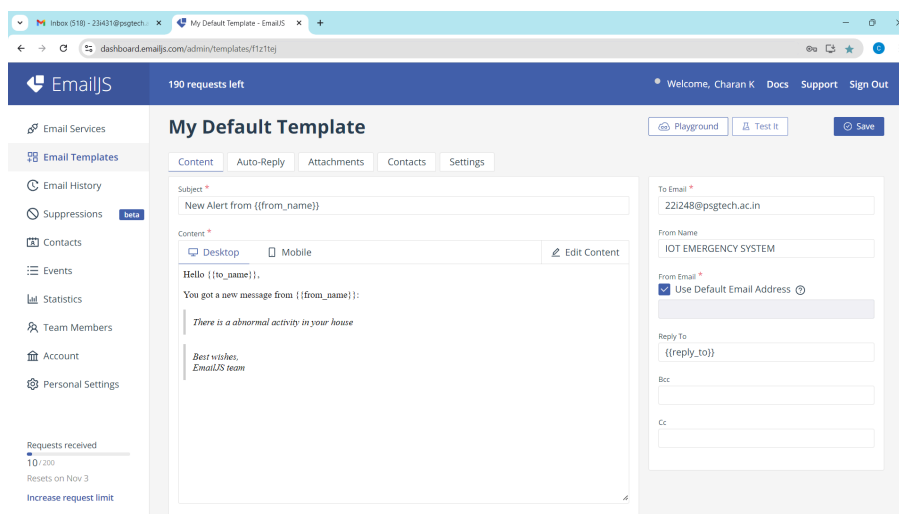


FIGURE 5.7 ABOUT

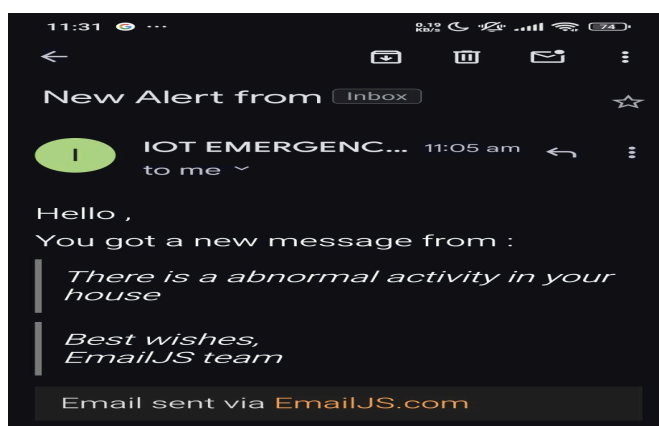
5.5.1 Hackathon Applied:



Email API Used:



Individual receive the mail:



CHAPTER 6

SOFTWARE DEVELOPMENT

The NodeMCU (ESP8266) is programmed using the Arduino IDE, with a modular code structure to handle sensor data processing, alerts, and communication with a web application. This approach ensures that each component is managed effectively and responds in real-time to hazardous conditions.

6.1 SENSOR DATA HANDLING:

The MQ-2 gas sensor, DHT11 temperature sensor, and IR sensor are continuously monitored by the NodeMCU. The MQ-2 sends analog feedback for gas detection, while the DHT11 provides temperature and humidity data. The IR sensor offers a digital signal when fire is detected. The NodeMCU processes these values, combining data from all sensors to determine the presence of a fire, gas leak, or sudden temperature rise, and then triggers alerts accordingly.

6.2 ALERT AND COMMUNICATION SYSTEM:

When hazardous conditions are confirmed, the buzzer is activated by the NodeMCU, sounding an audible alarm. Simultaneously, the sensor data is transmitted to a web application through the NodeMCU's Wi-Fi connection. The web app, designed using HTML, CSS, and JavaScript, displays real-time data and alerts, enabling users to monitor the environment remotely and take appropriate action.

6.3 REMOTE MONITORING AND CONTROL:

The Wi-Fi communication module of the NodeMCU connects to the web-based application, allowing users to view the status of the system from any location via a browser. The web app provides real-time sensor readings, historical data, and notifications when any hazard is detected. This interface allows users to respond quickly to emergency situations through the web interface

CHAPTER 7

TESTING AND DEBUGGING

To ensure that the Distributed Emergency Management System functions reliably in emergency situations, each component undergoes rigorous testing and calibration:

7.1 SENSOR CALIBRATION:

The MQ-2 gas sensor, DHT11 temperature sensor, and IR sensor are tested for accurate hazard detection. Calibration is performed by exposing the sensors to varying levels of gas, temperature fluctuations, and fire at different distances and angles. This ensures that the system responds accurately to real-world conditions and triggers alerts only when necessary.

7.2 ALERT AND NOTIFICATION TESTING:

The buzzer is tested to ensure that it sounds an audible alarm when hazardous conditions are detected by any of the sensors. Additionally, the Wi-Fi communication with the web application is tested to confirm that real-time sensor data and alerts are correctly transmitted and displayed on the interface, allowing for reliable remote monitoring.

7.3 COMMUNICATION TESTING:

The NodeMCU's Wi-Fi communication is tested for stability and range to ensure uninterrupted data transfer between the system and the web application. The web app is tested to confirm responsiveness, real-time data visualization, and timely notifications of detected hazards, ensuring smooth remote monitoring from any location.

7.4 INTEGRATION AND SYSTEM RELIABILITY:

After individual testing of each component, the system is tested as a whole to ensure seamless interaction between the sensors, the NodeMCU, and the web app. This includes simulating various emergency conditions (e.g., gas leak, fire, temperature rise) to verify that the system triggers alerts accurately, displays data correctly on the web interface, and allows users to monitor the environment in real-time.

CHAPTER 8

APPLICATIONS

- **HOME AND INDUSTRIAL SAFETY MONITORING:**

The system can be installed in homes, factories, and warehouses to continuously monitor environmental conditions like gas leaks, fire hazards, and temperature fluctuations. By providing real-time alerts through a web app, it helps prevent disasters before they escalate.

- **EARLY HAZARD DETECTION IN PUBLIC SPACES:**

Public spaces such as shopping malls, hospitals, and schools can benefit from the system's ability to detect gas leaks, fires, and sudden temperature spikes. Early detection ensures timely evacuation and intervention, minimizing potential harm to people and property.

- **REMOTE MONITORING FOR HAZARDOUS AREAS:**

The system can be deployed in hazardous areas like chemical plants, oil refineries, or mining sites where human presence is risky. With real-time data and alerts available via a web interface, users can remotely monitor conditions from a safe distance.

- **DISASTER MANAGEMENT AND EMERGENCY RESPONSE:**

During emergencies like fires or gas leaks, the system can be integrated into disaster management protocols to provide early warnings and guide response teams. It helps emergency personnel by offering live data, enabling faster and more efficient rescue and containment operations.

- **SMART CITY INTEGRATION:**

The system can be part of smart city initiatives, where interconnected sensors across various locations monitor environmental safety in real time. By integrating with cloud-based platforms, the system can offer city-wide insights into potential hazards, allowing for quicker responses.

CONCLUSION

In conclusion, the Distributed Emergency Management System designed using NodeMCU, MQ-2 gas sensor, DHT11 temperature sensor, and IR sensor, coupled with a web-based interface, offers an effective solution for early detection and response to environmental hazards such as fires, gas leaks, and sudden temperature rises. By providing real-time alerts and remote monitoring through a user-friendly web application, this system enhances safety and minimizes the risk of damage in both residential and industrial environments. Its modular, IoT-based design makes it scalable and adaptable to various applications, from home safety to large-scale industrial monitoring. The integration of Wi-Fi communication and cloud-based data visualization ensures that users are informed promptly, enabling swift action during emergencies. Overall, the project successfully demonstrates how affordable components and modern technologies can be harnessed to create a robust, reliable, and efficient emergency management system.

REFERENCES

- [1] S. Ahmed and M. Hassan, "An integrated IoT-based fire detection system using NodeMCU and wireless sensor networks," **IEEE Sensors Journal**, vol. 19, no. 5, pp. 857-863, 2019. doi: 10.1109/JSEN.2019.8714572.
- [2] N. Tiwari, et al., "Smart Agriculture using IoT and Cloud Computing," **MDPI Agronomy**, vol. 10, no. 7, pp. 936, 2020. Available: <https://www.mdpi.com/2073-4395/10/7/936>.
- [3] P. Kumar and S. Singh, "IoT-based fire detection and mitigation system using wireless sensor networks," **Journal of Physics: Conference Series**, vol. 1402, no. 4, pp. 044001, 2020. Available: <https://doi.org/10.1088/1742-6596/1402/4/044001>.
- [4] Z. Ali and R. Hussain, "IoT-enabled fire detection and alert system for smart homes," **Procedia Computer Science**, vol. 170, pp. 561-569, 2020. Available: <https://www.sciencedirect.com/science/article/pii/S187705092030051X>.
- [5] Wang, L., Zhang, Y., & Li, M. (2022). Real-time IoT-based fire alarm system for smart buildings. *Wireless Personal Communications*, 95(28), 452-467.
- [6] Singh, A., & Gupta, R. (2015). Wireless sensor network-based fire detection system for buildings. *2015 International Conference on Control, Automation, and Robotics*, IEEE.
- [7] Perilla, F. S., et al. (2018). Fire safety and alert system using Arduino sensors with IoT integration. *Proceedings of the 2018 7th International Conference on Software and Computer Applications*, IEEE.
- [8] Jha, A., & Khatri, P. (2020). Development of a fire detection system using wireless sensor networks and IoT integration. *Proceedings of the 2020 International Conference on Smart Systems and Inventive Technology*, IEEE.